

Active Inference ModelStream 007.1 ~ Conor Heins & Daphne Demekas ~ pymdp

Source: [YouTube](#) Playlist: ModelStreams Duration: 1:47:29 | Uploaded: Unknown
Transcript method: auto_caption

foreign this is the active inference Institute it's November 15 2022 and we're in model stream 7.1 we're going to be discussing Pi mdp a python package for active inference in discrete State spaces we will all say hello then we'll pass to Connor for a presentation following the presentation we'll have some discussion take a look over Pi mdp scripts take any questions that are coming up in the live chat so thanks to the authors for joining today and also to Jacob we'll just start by saying hello so I'm Daniel I'm a researcher in California and I'm really excited to learn a little bit more about pi mdp and see how active inference gets applied and I'll pass to jakub hi everyone I'm yakub I'm a student in the UK and also very excited to hear more about parent and discuss the recent uh developments and plans for future development I'll pass it to Daphne hi I'm Daphne um I'm working here in London I've used primary p a lot for work that I did with Connor on my Master's thesis and also just work that we've been doing since then so um I definitely like think it's a really really great package and I'm happy that people are gonna start using it more excellent all right Connor thanks a lot for joining take it away great thank you thanks for the invite I'm glad that uh we um arranged this it's like a nice opportunity to go on the live stream I've been on a few times now so it's always nice to come back so I don't have a slide introducing myself so I'll just say a quick sentence just about who I am so I'm Connor I'm a PhD student in biology at um the plug Institute for animal behavior and Constance in Germany and most of my work is about applying active inference and kind of the Bayesian lens on cognitive science and complex systems applying that to Collective Behavior like Collective animal behavior but today I'll be talking about kind of one of my side PhD projects which has been developing this Prime DP package which was very much a collaborative group effort with a bunch of people from the active inference Community um so yeah let's uh let's Dive In see the the paper that the package has actually been out for a while and the paper came out earlier this year um and it's just it I I want to begin by emphasizing that primevp is very much a work in progress even though we released a paper and it's

definitely like a usable Standalone package it's there's always a ton to keep developing and towards the end of the presentation I'll talk about some of those ongoing developments which are really exciting in my opinion and we'll really open up the the usage um and extendability of the package thank you so basically I mean this is an active inference Institute podcast or a live stream so I don't have to spend too much time motivating I don't think active inference but in in short the pi MVP package is a python package for simulating um and running active inference processes in discrete State spaces and the discrete State Space Case is a very well-studied case um and very well characterized and it's been very popular in like models of decision making like discrete decision making and planning and has seen a lot of application in the neurosciences as models of like um discrete decision making behavior in for instance humans or other animals um so just I'll give a little outline of the presentation so first I'll just introduce the team of uh people who have worked on primevp and we're authors on the paper but the actual effective team goes much larger than that because there's a lot of people who are who are developing or using it and contributing in their own ways that weren't actually co-authors on the paper um and then I'll discuss the motivation for the package I'll give a brief overview of active inference in discrete State spaces but as I mentioned I think this is a really nice like venue to have this discussion about Palm DP because I don't think I need to go too deep into discussing what active inference is so that'll save us some time to like get more into the depth of pond DP um I'll talk about the existing approaches to simulating active inference agents like namely in Matlab using SPM and just kind of compare and contrast Prime DP with SPM and and help us understand like kind of where Prime DB is coming from um in terms of its origins in SPM really so then we'll talk about some of the features of prime DP and its General package structure and then we'll dive into a few usage examples where I'll show some like outputs of simulated agent behavior and then the code that accompanies that just to kind of demonstrate what the general flow of Pi mdp looks like and then at the end I'll just discuss some of the future directions and ongoing um active branches I guess of pine DP their ongoing development efforts that I think will make time EP super exciting and extendable um to all kinds of new use cases so I'm really excited about that and now and as Daniel was saying if at any point someone wants to um but in or it has questions for clarification just feel free to let me know and we can kind of dwell on some points for longer uh okay so I'll start by introducing the team so they're the co-authors in the paper Edition to me were Baron millage Daphne to make us who is here today uh Brennan Klein Carl friston Ian cousin and Alexander chance so Carl and Ian are my co-phd supervisors and um Carl is the original progenitor of the

mdp or discrete State space formulation of active inference in Matlab um so it's nice to kind of have his uh stamp of approval on our work here and he's kind of cosine on that and then just I I think it's important to emphasize how critical everyone here is to the package and it wasn't really just I did do I guess most of the actual software development but the early stages of prime DP were really conversations between me Brennan and Alec back in 2019 or maybe even earlier just about the need for a python package that does active inference and I think probably other people were having similar conversations around that time but it was because we all kind of came together that we're able to to build this thing in not too long of a time I mean more than two years and it could have probably been done faster if I was working full-time on it but it was um it was really fun to kind of watch the progression of this and then as Daphne said she's actually one of the the first people who really used um active Pi mdp in her own Master's thesis and not only that but in a very ambitious application which is like multi-agent Collective Behavior having a bunch of pine DP agents interacting with each other to simulate the kind of opinion Dynamics and Echo Chambers and social networks which is really exciting so that was really gratifying to work with her on that and um yeah so it's really nice also that Daphne is part of this because it's like a very good example of a Pioneer in the pi mdp active inference community and then Baron also uh me Alec and baron did a lot of work on developing some of the more sophisticated message passing techniques inactive inference in in Pym DP and uh Baron was also really critical in helping me write the paper um and he's just great at writing and conceptualization and just was also used actually primevp in some of his own work on successor representations and active inference which is kind of cool okay so motivation one of the biggest things which everyone here is I'm well aware of is there's just much more popularity and interest in active inference these days in the past 10 years and especially the last five years um so it's kind of obvious that we need some more General user-friendly Frameworks for actually letting people learn about active inference first of all from a pedagogical perspective as well as applying it in their own research or industrial applications or whatever they want to actually do with active inference um and there's a lot especially of interest in active inference from certain communities not just Neuroscience anymore but things like machine learning data science engineering Network science even software development software engineers and a lot of those interested Fields um the most dominant uh language is python so when a lot of times what I've heard is when people come to learn about active inference and they're have a background in python or r or something and they figure out that it's all in Matlab they have a there's kind of a barrier to entry because they either don't

know Matlab or they have a hard time parsing Matlab code especially if they're from like non-array programming Frameworks like maybe they're a front-end web developer who uses uh JavaScript or something and they're not actually familiar with like big multi-dimensional array programming um so that was another motivation for making specifically a python package that does active inference um and then finally we because it's in Python that means that we're now creating an ecosystem that can talk to other ecosystems so ideally primevp isn't going to be just used in a script that only uses Pi mdp it's going to be used with other software packages some of them having to do with artificial intelligence or network science or all kinds of Frameworks that are relevant and so you can now kind of plug and play with piondp agents and put them in environments like open AI gym for reinforcement learning for instance and uh and now you can work with active inference in diverse applications and that's really great for python because python by Design and by its Community Driven development just has so many different packages that were built very well for some specific thing so now that it's in Python you can kind of uh use it with in tandem with all those other python packages um okay so now a brief intro to active inference so the fundamental Paradigm of active inference is that you consider an agent embedded in its environment and unlike kind of traditional more passive approaches to uh perception and behavior where you kind of consider the environment gives you information you do some sensory motor transformation and then you perform an action active inference very much um emphasizes the fact that inference or the problem of dealing with uncertainty characterizes both perception or what we call like State estimation or belief updating as well as action which is where the active inference part comes into play so agents are not only updating their beliefs about the states of the world the hidden States out there by minimizing this um bound on a surprise called free energy and that's where these kind of ham holsian ideas of perception as inference come from but also you're minimizing surprise or abound on surprise also to choose your actions so inferring actions becomes just another another sort of inference problem so policies or sequences of actions are considered latent variables or hidden States and then you also do inference about those and by casting both sides of the perception action coin as a example of surprise minimization what you events are agents that kind of display purposeful and curious behavior and there's very like there's nothing about active inference that means you have to use discrete State spaces or Pi MVP that's just one particular type of or class of generative models for active inference but um uh the the Palm DP discrete State space generative models are really easy to work with with active Insurance because a lot of these quantities that I'm showing here are very easy to compute when you're

dealing with palm DPS or partially observed markov decision processes so we'll get into all the mathematics in a little bit but that's just a basic Paradigm of of active inference agent and environment Trying to minimize surprise both doing perception and action by minimizing surprise and just for a more in-depth mathematical review of active inference and discrete State spaces namely using these partially observed Markov decision processes I would recommend reading this paper which is excellent in the Journal of medical Psychology by Lance Thomas Noor Sebastian victorita and Carl it's just a really great description from a formal basis of how we get to the update equations for active inference starting from the most like formal treatment of categorical and dirichlet distributions and also the pine DP paper the version we have on archive also has a bunch of appendices that do a lot of this similar sort of math so I would also refer people to that okay so now let's get into the generative models that form the bread and butter of uh the the agent's brains in Pym DP essentially so Central to active inferences writing down a generative model which is just a specification of how an agent believes its World Works how does it its environment um how does it believe its environment influences itself like the Dynamics of the world uh progress and how does that those hidden State Dynamics also give rise to observations that's all encoded in what we call a generative model or a world model some people also call so in Pine DP we only deal with a very specific sort of gender of model which are called these partially observed Markov decision processes so these are a classic Model of sequential decision making and planning under uncertainty they're not unique to active inference people use Palm DP models for classical reinforcement learning and all kinds of decision-making problems um they're called markovian Markov decision process because the state at the current time only depends on the action in the state at the previous time so that's the definition of a Markov process they kind of have this shallow temporal dependence um that for instance a non-markovian process doesn't have like a process that has a longer term or deeper temporal dependencies that stretch further back in the past um Palm DPS are often but not always formulating discrete State space and discrete time there's nothing about the word palmdp that means they have to be these multinomial or categorical distributions but just when we talk about Palm DPS and active influence we're almost always talking about these discrete ones um that have to do with basically you can only be in one of K discrete States at a time and you only progress to one of K discrete States at the next time so everything's discrete but there's nothing intrinsically about these Markov decision processes that has to be discrete I think it's just worth mentioning that because that's a important kind of conflation that people often make when they see the word

Palm DP um and the reason we decide to use this uh discrete Palm DP generative model for active inference is not just because of the applications to sequential decision making and planning but also there's just a massive pre-existing active inference literature since 2010 2011 on using pom DPS as generative models for decision making tests so all the mathematics for doing active inference with these models is already done so we didn't have to invent any new math or Theory to actually code this up because a lot of it has already been written in papers for like 10 years so that made our life easier When developing it so now um let's dive into just the main components of these Palm DPS uh there I'm only listing the four major ones here but there's other components that we can discuss if anyone's interested but essentially this this line at the top is a description of the generative model in terms of a joint distribution over hidden States s and observations o in Into the Future so uh the because of the markovian nature of this generative model you can write um The Joint distribution with this factorized basically product of um priors and and likelihoods that factorizes across time so that's why they're those products over time in the top but it doesn't matter understanding the the math right now but the main components which we'll map on to a schematic in a second are the agents beliefs about how the hidden States cause observations which we encode in something called the observation model or the likelihood mapping often called the A matrix or the A array so this is a probabilistic representation of how do hidden States at the current time affect or give rise to observations at the current time second we have the transition or Dynamics model which is another sort of likelihood that encodes the agent's beliefs about how hidden States at one time relate to Hidden States at the next time so this Maps um this is what the agent uses to make forward predictions about how will the world evolve if this happens or if that happens as well as to carry kind of messages or empirical priors from the past given where I was yesterday where must I be now given my my beliefs about how the world evolves sometimes that's all encoded in the B array or the B Matrix and then these final two are kind of uh priors the first of which is very important which is called the C array or the C Vector which encodes the agent's prior beliefs about What observations it's likely to encounter and as we said in the beginning active inference is all about casting both action and perception as an inference problem so active inference as a framework kind of turns the classic Paradigm of reward functions and reinforcement learning on its head by saying instead of a reward function just equip the agent with a kind of optimistic belief that in the future I will see this kind of data and then by performing inference with respect to a generative model that has that prior belief in it the agent will look like it's searching for observations that conform with its priors so this goes

in line with this idea that active inference is a process of self-fulfilling prophecy the agent believes I'm more or less likely to see certain observations and then by doing inference about policies with such a bias to generative model the agent will kind of bring itself to actually realize its prior preferences or these Pro these prior beliefs about observations so that's all encoded in the C vector and the C Vector you can basically think of as the Bayesian translation of the reward function and you can actually exactly relate the entries of the C Vector two rewards and reinforcement learning but we won't we don't have to get into that now but I can share papers if anyone's interested and then finally there's the prior over hidden States and this is simply a the agent's belief about what is the prior likelihood of each hidden State at the at the first time step of the simulation so this isn't really a necessary thing in active inference but oftentimes when we're simulating agents we we use a finite temporal Horizon with a start time and an end time so if you have a like a finite temporary Horizon it means that you have to basically plug in a prior belief about what the world looks like at time step one and that prior belief is encoded in this d d vector and we can just intuitively sketch out the the Palm DP generative model as a Bayesian graph where nodes are connected by edges by these arrows if they depend on each other so these red nodes represent the hidden States transitioning to each other over time um and the blue nodes the O nodes represent the agent's beliefs about how observations are generated from those hidden States so that's just a graphical representation of the Palm DP and the a matrix encodes as we said the agent's beliefs about how those red nodes give rise to the blue nodes at any time so usually we assume that this a matrix is time invariant so they they believe that this observation mapping doesn't itself depend on time that's at least that the Assumption made in most Palm DPS and then there's also the agents time invariant beliefs about how the transition dynamics of the world uh bring states that time T minus one two states at time t en and then policy selection or action comes into play in inactive inference by Framing actions as controlled transitions so the B Matrix does not just say how does it how should the world look now given how it was yesterday but how should it look now given how it was yesterday and the fact that I took this action at time T minus one so the B Matrix is not only conditioned on past States but also on previous actions so policies P_i come into play as a belief or a distribution over those actions so the actions directly influence the Transitions and then the transitions affect the states so that's how kind of we think about action in a lot of palm DP scenarios and actions are or policies P_i are sequences of actions or collections of actions and then we have this critical objective function uh called the expected free energy that determines which actions are more likely than others so by minimizing expected free energy

we optimize a belief about policies and then the expected free energy itself is a function of your generative model your beliefs about the world which includes this biased prior belief about What observations you expect yourself to see so the kind of reward function enters policy selection via this expected free energy and that's why we're able to still kind of call all of this a form of an inference problem because we're minimizing this expected bound on surprise so and in doing so we kind of infer distribution over policies which we then sample from to actually generate actions and change the world and then as I said the D Vector is basically just a prior on that initial hidden State distribution so yeah in summary to build a palmdp active inference model uh you have to write down or encode these a b c 's and D 's and there's some other priors too that we can talk about like a prior over policies which is called an e Vector but for now you can just think of most of the heavy lifting in active inference as consisting in writing these things down actually encoding what your agent believes about the world that it exists in um and these things in the case of these categorical discrete distributions they end up looking like matrices and vectors since everything is categorical distributions you you're not dealing with continuous infinite dimensional spaces you're dealing with things that have a discrete number of entries like a four by four Matrix or five by five Matrix uh yeah I think before we yeah so the next part I'm going to talk about like the Matlab python um dialectic before we move on to that though should we maybe pause if there's any questions foreign yeah great thank you Daphne or Jakob do you want to provide any thoughts or reflections there's one thing that I've always been a little bit confused about which might be nice to resolve now which is in that parameterization of the prior preferences can you explain like why exactly it is that when you initialize them you just do them with like um integers greater than 1 are greater than or equal to one but then you like soft maximum later on when you actually using an inference like why do we not Define our preference of those probabilities yeah that's a good that's a good um point and a good thing to mention so in usually in both the Matlab and the python implementation so when building Pi mdp we made the decision to to do the same thing when you're encoding that c Vector instead of encoding it as probabilities which It ultimately should be what you really write down is the log of the C Vector so you write it down in terms of relative log probabilities and the reason that's potentially a more more intuitive parameterization rather than writing it down directly in terms of probabilities is because the log of the probability is more like the actual reward so if we go back to this generative model the expected free energy being a free energy is actually and then a free energy is sort of a kind of KL Divergence it's encoded in terms of bits which is naturally

in a logarithmic space so if you're writing down like a prior in terms of in in log space you know kind of if I change the number of log units in my prior preferences I have it it's a it's more linearly related to the change in the expected free energy for a policy that leads to that observation whereas if you're writing on the priors in terms of a probability space that the ex the resulting change in the expected free energy from changing something in probability space is not going to be um linear it's going to be a non-linear change so if we think of rewards as log something that's like one extra log unit or one natural log one Nat or one bit more valuable than another thing that will actually reflect itself in terms of the expected free energy difference between seeing Thing One versus thing two whereas if we encode those two things in terms of probabilities that expected free energy difference won't be as intuitive simply because it won't be linear um so that's like a quick reason for why we decide to encode things in log space but there's nothing mathematically necessary by any means you could easily just uh write the priors directly in terms of probabilities it's more of a it's like a it's just a way to make it more reward like really yeah that makes a lot of sense thanks thanks Jacob yeah um I'm wondering is there um heart requirement on the amb matrices or tensors to be actually encoded as categorical matrices or because thinking about like the function of the a matrix and the way the agent uses it to infer its hidden state from an observation I'm thinking whether would be possible to encode High dimensional State spaces in something like a neural network that can approximate uh these probabilistic representations as well uh and learn the represent the kind of the relationship between observations and hidden States yeah definitely Yeah so basically sorry there might be a like but yeah basically whether you think that would be possible and because I I feel like the python implementation which I'm sure you'll get into the advantages of python over Matlab uh also affords the interoperability with other libraries and within data science totally yeah that's a good great point so I kind of mentioned towards the beginning when we talked about Palm DPS there's nothing about Palm DPS that means all these things have to be matrices the main thing that defines upon EP is that it's partially observable so that the the agent only gets to see the blue nodes and that it's a markovian process so that there's like a temporal shallowness in the memory of the Hidden States but you could replace a and b with any kind of parameterized it has to be a proper likelihood function so it has to be have that like um proper the properties of being a likelihood there's nothing that means that it can't be a multivariate gaussian or koshy distribution or Bernoulli distribution or uh yeah any kind of exponential family or some like um parameterized neural network like that people use oftentimes in in reinforcement learning like they'll parameterize a Dynamics

model just with a bunch of neural networks that say take time State at time T and move them to State at time t plus one the reason there's difficulties with that is simply because it's unclear how to compute things like the expected free energy once you start moving into these um not as nicely behaved distributions so a lot of the reason that this framework was developed in the discrete categorical State version is not only because for modeling low dimensional tasks like someone playing a slot machine or deciding to go left or right in a maze it's easy to use these discrete distributions to kind of give a good description of that behavior that's one thing but also is literally a mathematical tractability reason there aren't always closed form solutions for the expected free energy depending on what those those distributions look like so there's some work on doing this for instance Magnus Cudall has a paper in entropy where they compute the expected free energy in linear dynamical systems basically where all these A 's and B 's are represented by gaussians so they they call this like a linear controllable dynamical system and they also have to make some assumptions about how the actions affect the B Matrix which really isn't a matrix anymore it's more like a continuous gaussian and and they find that the normal terms you get with the expected free energy like the Information Gain those a lot of the interesting things that we get with when we use categorical distributions those terms actually disappear when we use gaussian so you actually don't get the nice like Information Gain seeking or information seeking terms that you would normally get by using categoricals but there's other ways to get around that like if you use neural networks you can still use things like sampling approaches to compute expected free Energies um where you basically like sample a bunch of possible trajectories and then you can use the samples like kind of Monte Carlo style to compute expected free energies and a lot of the groups that have done um like scaling active inference to deep neural networks they've used that kind of approach so like Alec Chances paper Scout scaling active inference does stuff like that a lot of the stuff out of like Tim Verbalance group um I don't want to leave anyone I think like in nor has done obviously a lot of North Vegeta has done a lot of active inference with deep neural networks as a virus found us like a lot of these people were actually trying to apply active inference to keep neural Nets they have to basically come up with ways to compute that expected free energy that are different than the way it's done in pi MVP because in Pine BP you can exactly compute that expected free energy which ends up just being a bunch of Matrix Vector products and then a summation so it just becomes more difficult when you use more complex distributions but it's by no means impossible you just have to you have to come up with some kind of approximation but there are ways I think you can get around that which

involve what I call what people have been calling Hybrid models where you might have deep neural networks that feed into a palm DP at like some higher layer and then the impound DP space like a discrete categorical space you can still compute expected free energies and free energies but you can still take advantage of at lower levels like high dimensional neural networks to project your data into this low dimensional space so you do all the efc active inference in this lower dimensional Palm DP space where everything's exact but you can still take advantage of the nice dimensionality reduction and feature extraction properties of neural networks to first like pre-process the observations so that's something I'll talk about at the end because we've made some progress on the first steps towards making that possible which involves basically making all of Pi mdp Auto differentiable so you can like train neural networks that are attached to pump the prime T models yeah it's a great question though awesome I see this as like the body plan of generative models and maybe what's it like today is an antenna tomorrow or delay it gets longer or thicker there's a lot of different ways to swap out and compose different parts of the model so thanks carry on cool yeah so yeah now let's talk a little bit about the traditional software for active inference research which I'm sure everyone on the call right now is familiar with the original way it was done was basically uh Carl Friston and a few others wrote a bunch of Matlab scripts part of the SPM package just originally go out for like neuroimaging data analysis and statistical testing and there's a sub package called Dem or dynamical expectation maximization which is used for not only active inference including continuous active inference but also just like generalized filtering and fitting non-linear State space models from empirical data So within that we have all the this kind of discrete active inference toolbox most of the functions of which are prefixed by SPM underscore Mt mdp so the main function that basically does everything that Pine MVP does for the most part is is containing one function called SPM mdp vbx and kind of in in a tongue-in-cheek way we often joked around that Pine DP is just a package that implements that one function because that one function essentially does active inference and learning and all the message passing just in in one call which in many ways is very nice because you just have to pass in a mdp to it and then it in a kind of Black Box way gives you all the belief updating and history of actions and everything that that you need um there are despite its elegance and it's like usefulness and how robust it actually is because you can pass in any agent with any generative model and it'll pretty much work the issue is because it's just a single function it's not very modular so it's very hard to flexibly compose different subcomputations of an active inference process so if you want to do some bespoke active inference application where like

oh um before I in and do hidden State inference I want to update the parameters of the a matrix and I want to do it in this particular way it's very hard to do that in SPM mdp vbx so what often I see happening and I did this myself in my Master's researches I end up having like five different versions of this function that have some little like specialist prefix for the the specific thing that I was doing and then that just ends up being like uh kind of not not efficient because you're creating a bunch of boilerplate code then you have one function that does the particular little version of active inference that you want to explore in your project so just by modularizing this one function even within Matlab you could do this you would uh basically make the thing a lot more flexible and save a lot of copying and pasting of code um another issue is that inference and policy selection are fixed so you can't kind of compare and contrast different message passing approaches and action selection routines so there's just one way it's done in SPM vbx which is called marginal message passing um and so it's hard to like compare that like what if you change the message passing algorithm whereas in uh Pi MVP we have it right now two message passing algorithms that you can kind of side by side compare I mean we hope to add more in the future which is very easy because it's just a modular thing and then because of what we were talking about in the beginning with it just by virtue of this being in Matlab it's harder to synthesize with other Frameworks like in reinforcement learning open AI gym and uh deep neural networks like tensorflow flow or Pi torture Jacks so just the fact that it's in Matlab um already limited of course people have found ways to do like cross language cross-platform um code like moving from Julia to Matlab to Python and back but it's just a lot a lot more heavy lifting involved in that and then I'll just lay out some um kind of pros and cons of Matlab and python so one of the things I do like about Matlab is it's very easy to get started with array programming has a nice editor there's a lot of use and history and computational Neuroscience like even when I gave this tutorial at a course on computational Psychiatry this past year I think the majority of the tutorials were still using Matlab so there's still a good reason to use Matlab just because of the history and the number of packages that are well suited for doing neuroscience and psychophysics stuff so there's that Advantage just kind of like a almost momentum or a legacy advantage but of course there's issues like as proprietary I think that's the biggest one you need to pay um One Way or Another to to use Matlab and there's not as much Community Driven development I mean there is some like the file exchange but it's not the same level as something like Python and then python kind of can compete with Matlab because it has array programming in the form of numpy of course it's open source it's been um widespread adopted across tons of fields not just

academic ones but lots of commercial application and there's a lot of community give driven development in the form of packages like Pi VP um but one disadvantage is for me at least it wasn't as easy to get started with Matlab um as it was with Matlab because you'll often have to like install a bunch of things and learn about virtual environments and you have to like learn about a lot of more programming stuff before getting started with using python so I think that is one reason that Matlab is actually still useful as a pedagogical tool like the the time between getting Matlab and actually doing programming is is pretty short which is nice um so yeah the package is called uh it's within this infra actively GitHub organization it's called Pi MVP and you can just pip install it like in a virtual environment or just in your base installation of python and then once you have a piondp installed you can kind of uh import and use in a flexible modular way all the different sub packages or sub modules like the agent module which basically just implements the agent class and then there's different modules like inference and control and learning that are independent of actually an active inference agent and you can just use them to do message passing for hidden State inference or to compute expected free Energies on possible policies or to compute updates to parameters so all those things can be composed and flexibly kind of you can make kind of Frankenstein active inference agents by composing all these things in in the bespoke way that you desire and a lot of it um the main workflow of pione P comes down to specifying the generative model and the forms of these discrete arrays and then plugging them into the brain of the agent and instantiating an agent so that is pretty much encapsulated by these two lines at the bottom import the agent from Pine BP and then you just build one by plugging in the ABCs and D's and then you have this agent object that you can use to do hidden State inference through methods like in first States infer policies which is the agent internally computes the expected free energy of its policies and then you finally can sample actions so these three lines are the the kind of the main players of any active inference Loop State inference policy inference and then action selection and you just kind of tie those in a loop over time um to to instantiate an active inference process and the the kind of um the circularity of the action perception Loop comes into play with the inputs to the agent which are its observations and the outputs of its action selection where charts are its actions and of course you need to use the Last Action to then get a new observation and that you you do by plugging the action into some environment this is also in the active inference literature often referred to as the generative process so the actual world out there that generates your data so this is the kind of classic action perception group you'll see enacted any active inference agent and this is also not even unique to active

inference this is just how reinforcement learning problems are generically framed so like open AI Jam uses a very similar um control flow we're kind of like um yeah sensory motor Loop so yeah here's an example of just um doing that like import the agent set up the generative model and that's the hardest part so I kind of conveniently put ellipses after the dot dot but that's actually where most of the code is going to happen is building the generative model build the agent you build some environment which you can either import as a one of the stored Pi mdp environments or you could get it from open AI gym or you could just create your own environment so these this would be your code that actually describes how the world works out there the world that the agent is interacting with and then you can Implement a Time step of active inference with those few lines of code right there and that would all be kind of those last lines like 8 through 15 those would kind of be wrapped Within a loop over time um yeah this is just more examples of like this would be a quick way to in this example we're not even running active inference but we're just using one of the message passing algorithms from the Algos sub module to do hidden State inference so in this case I just created a random a matrix I created a random observation and I gave and I made up a random prior and then I can just do one like fictive update of what an agent might be doing during hidden State inference and optimize Qs or the beliefs about hidden States so this is an example of how you could actually use Pond DP to just do generic inference on hidden Markov models you don't even need to use it within an active inference Loop you could just use it to do uh like statistical inference on a hidden Markov model with the particular algorithms that we've provided and then of course one of the advantages I talked about over the SPM is that you can create customized active inference processes there's a lot of like extra arguments to the agent class where you can kind of turn on and off different parts of the reward function or the expected free energy for an agent so for instance in this agent the agent does not incorporate uh expected utility which is the kind of reward C Vector driven component of action selection but the agent does use State info gain and parameter information game which are two other components of the expected free energy so there's a lot of ways that you can kind of create a bespoke active inference agent that doesn't that will behave in different ways depending on these kind of uh keyword arguments that you provide yeah here are just more examples like in this case we're using active inference agent just to do hidden State inference and no actions at all so the agent is just inferring hidden States it's updating its beliefs about the a matrix and it's updating its beliefs about the D vector or the initial hidden state and so you can you know just flexibly put together all these lines and create an agent that does whatever you want doesn't even need to

be acting in the world technically okay so now I'm going to show a few examples of pine DP in action um so this is one that's a classic active inference paper it's one of my favorite papers it was um I think it's called scene construction and active inference where it's describing a task where an agent has to and this was used to actually model human data in a psychophysics test an agent has to gaze contingently uncover two out of four quadrants that have particular images in them um and there's four quadrants total and two of them have uh the images of interest in them and if the two images in this case are a bird and a cat image and that's an example of a flea scene so the Asian basically or the human has to categorize the latent scene which is simply defined by the combination of two images in order and then categorize the scene so it's basically a categorization categorization task but the agent needs to gaze contingently uncover a sequence of cues before it knows what that category is or what that scene is so it combines the epistemic components that we all know and love about active inference trying to uncover the hidden state of the World by actively sampling it so in this case they're sampling the World by moving their eyes to different quadrants to uncover what's what's behind them and then actually choosing what the the true category is based on what it learned about the world and that's where it's trying to maximize utility because there's some reward associated with categorizing correctly so this is an example of that which was originally done in Matlab and I just re-implemented in pi mdp and this is another example where now the the scene is um the feed scene so the the cues are in the lower two quadrants in this example so the agent has to look around the different quadrants it finally sees that there's okay there's a bird in the lower right seeds in the lower left so this must be the feed scene um and just uh to show an example of what does that actually look like in active inference like what does the code look like for that so the first thing you would do for this one is you would set up your agent which again you just throw in the a b c's and D's in this case I used a particular message passing algorithm called marginal message passing which is the same one used in Matlab you set up a policy depth and an inference Horizon which is kind of like a memory like how much of the past observations you take into account and then you set up a environment this is like the external world that the agent will be interacting with in this case I'm calling it The Scene construction environment which just tells me once the agent moves its eyes to a certain place how does that actually then determine what the agent sees next which will be you know whatever is behind the quadrant that it decided to look at so those are the main two things those are the two sides of the action perception Loop the agent and the environment and then what it's often common to do is to get in an initial initial observation by

resetting the environment which is a convention borrowed from openai gym you basically do environment dot reset it's like a method of the environment that spits out the initial observations and then it's often useful in these things to create like lists or dictionaries that have a mapping between the observation indices which are like integers between zero and however many observations there are and then what those actually correspond to semantically so this is just a very common way to like because all the Palm DP and the environment will be spitting out are like ones in twos and zeros and all these like discrete indices but it's useful to have these lists that you can use to kind of semantically map particular indices to um things that are meaningful like seeing the bird image or choosing um the category one versus category two and then once you've done that you just write a loop over time where you're basically performing active inference which consists in Hidden State estimation and policy inference you uh sample an action which then gets fed back into the environment to produce another observation and then that happens over time so that's the whole action perception of so this is um yeah so that's it's deceptively simple how like short it looks but I'm kind of glossing over something that I'll talk about later which is I mentioned earlier which is as easy as this looks the hardest part is actually done way before any of this happens which is writing down the a b c's and D's that's by far the most time intensive and like complex part of active inference is actually writing down the generative model once you have the generative model written down then the rest basically is like clockwork you just have to link the agent to the environment and then just run like five or six lines just to actually implement the thing but the hardest part is writing down those a b c's and D's in the beginning um I'll show another example which I kind of uh call teammates on steroids so in the classic teamaze task um that I forgot what the original paper was but it's something that's been very popular in the active inference literature for a while uh that I think Carl came up with maybe in 2015 or earlier even is you have an agent a mouse that has to visit two potential sources of either reward or punishment in its environment and it doesn't know which arm of this teammate contains the reward so it has to visit a cue first before it knows which arm has the reward and which one has either no reward or like a shock like a negative stimulus in this one I just kind of spatially extended the teammates so the agent has to now visit a sequence of cues Each of which reveals the location of the next cube in order to figure out the final cue which is just the location of the cheese versus the shock so this is another example where the agent first goes to q1 then it knows where Q2 is and then it knows where the cheese is so I call this epistemic chaining because the agent doesn't actually have to plan its its root all the way to the final uh

location of the cheese all it has to do is get to the next q which then reveals where the next cue is which finally reveals where the hidden location of the reward is so you're kind of using epistemic value or curiosity to allow an otherwise temporally shallow animal to plan its way to um a distal reward or something that's that it can't plan to get to um *a priori* and again this is just an example of what that would look like in Pine DP where it basically looks exactly the same it's just that the environment and the generative model are different but the general flow of the code is always has this this kind of classic pipeline um and okay so now I'll get to the kind of the most important part and I think the biggest source of confusion um with active inference is that the hardest part is the generative model it's all the um complexity comes into encoding the agent's beliefs about the world so how do I write down the A's and B's and C's and D's in in paradigms like deep neural networks or like unsupervised learning you don't have to write down the model the neural network learns the model by just observing loads and loads of data so it's less sample efficient but you don't have to encode as much to begin with so this is kind of the kind of coincides with a larger divide between model based approaches with deeper neural networks you're you're effectively issuing the sample the statistical complexity of having to write down the model by just sticking together a bunch of non-linear function approximators and then just learning the beliefs that the agent has about the World by just bombarding it with data the same thing goes for deep reinforcement learning like DQN learning in active inference the agents are much more sample efficient in the sense that they don't need to train on like billions of data vectors but on the other hand there's more investment on your end as the modeler because you have to write down explicitly what the agent's beliefs about the world are you don't just equip it with something as generic as a convolutional layer and some reviews and stuff and then let it learn you actually have to hand code that so I think this is one of the biggest differences between model based reinforcement learning where you actually encode a Bayesian General generative model of the world and kind of more model free or data-driven approaches but there is it's not such a dichotomy there are ways to kind of combine the two but just to kind of show um that very specifically for instance in this scene construction demo that I showed like a few slides ago if you just look in terms of sheer lines of code which one took more code you can kind of use the the amount of lines of code as a proxy for statistical complexity or how much information is contained so the simulation itself running the active inference Loop was like 15 lines of code just like and and they and that code itself is already very generic and not specific to the scene construction demo writing the generative model itself that's where all the heavy lifting is done that's where

all the information that's specific to that task gets encoded so for instance I just look at how I created the a matrix the beliefs about the observation mapping for the scene construction demo that already is like way more code than just running the entire active inference simulation so just like by the the sheer amount of code you can already tell oh yeah there's a lot of assumptions and and information that's being baked into the generative model and that's where most of the heavy lifting of active inference actually comes from um yeah so I just think it's important to remark on that because that's like a really key thing that I think anyone who wants to start working with active inference models in in discrete State spaces should kind of wrap their head around is that the the model does most of the work for you the expected free energy yes is a very interesting objective function that has many advantages but most of the power of active inference comes into writing down what your agent's beliefs about the world are and then once you have that then all the rest kind of just does does the work for you because the pi NDB code is very generic what's not generic is how you encode the beliefs about the world um okay so now I'm kind of finishing up maybe we should dwell just on that bit for a second does anyone have thoughts or comments or questions about this if not I can just proceed and finish off Daphne or Jacob or I'll ask one I don't have any questions all right well you've emphasized the specification of the agent's generative model and how about the other side of the coin how do we specify a generative process how do we specify the environment for the agents that's a really good question yeah basically everything I said about the um kind of applies to the generative process as well except the agent's interesting Behavior yeah I mean you can you could think of the generative process as driving a lot of that too I I guess the bottleneck is the generative model because if you create a really complex generative process so a really complex environment that has all kinds of fancy non-linear Dynamics but the agent's model of the world is super super simple so it just believes that there's you know a light switch that's either on or off then the possible Behavior you can get from such a simple agent is is limited by the complexity of its generative model so a very complex generative a very simple generative model will still not show very interesting Behavior even if it's embedded in a complex generative process but the most rich Dynamics will obviously happen when you have both a complex generative process and a complex generative model so all the work in um in building the generative model which I would say is the first line up here can also be matched by a lot of work in generating the the generative process as well which in this case is this epistemic grid World environment which is just a set of rules that says when the agent is in the queue location show them the queue identity like this one is relatively simple

but one interesting thing to think about and I'm sure like you Daniel have thought about this when it comes to um you know your work on active inference and and Collective behavior is the interesting thing about multi-agent Behavior is in that case the generative process are the actions of other agents so the generative process my generative process are actually the outputs of another active inference agent so that's one of the most complicated things in what Daphne and I had to Grapple with when we're doing the and and Mao as well Mao was the first author of the epistemic communities work this like social network Echo chamber stuff in that context the generative process is a little bit more difficult because the the process itself is consisting of other active inference agents that are also acting so the control flow of that code will look a little bit different where you're going to have to Loop over all agents get actions from them and then use those actions to parametrize the observations for all the other agents I mean that's just a generic statement about multi-agent simulations in general but it's uh it's particularly interesting when you think about agents trying to model other agents because almost necessarily every active inference agent will have an impoverished model of how the world works when the the way the world works is a bunch of interacting active inference agents so you're going to have to kind of necessarily equip each agent with a more simplified generative model unless you want them to all have like infinite recursion depth and be able to simulate in their own generative model the generative models of every other agent so yeah that's uh that I mean that was kind of a tangent about the multi-agent case but I think it's just an interesting um interesting complex to think about the the tension between the the generative model complexity and the generative process complexity and how they kind of mutually constrain the behavior of each other um okay shall I just proceed uh so yeah the last two slides I think is the kind of exciting stuff so here are a list of things that we'd like to do with prime DP in the future um I'll just go through them and then I'll dwell on a few that I think are the most important so one is fitting Pine DP models to empirical data so I've interacted a lot with people from the computational Psychiatry Community who are interested in actually creating models of behavior often human behavior um that are their Palm DP active inference models and one of the biggest I think limitations of pine DP right now is that people can't use pi mdp to infer the active inference parameters of like a human subject that's performing some task that's what you can do in SPM right now but unfortunately you can't do that in Pion VP so this is like really high in the priorities list and I think this is what will help primary P actually become competitive to SPM for the communities that are interested in fitting Pi mdp models to data so these are like kind of more empirical scientific disciplines like computational

psychiatry um other things is I think we need a better interfaces for actually generating and constructing generative models right now all that code involved in Building A and B matrices that really becomes the bottleneck for anyone trying to do active inference and in large part because constructing those arrays for complex generous models can be a real headache you have to do all this weird multi-dimensional indexing because like if you have like a bunch of different interacting variables in the world you have to create massive multi-dimensional arrays that have different numbers of extra Dimensions that correspond to all these possible contingencies in the world it kind of becomes a massive lookup table that you have to encode all the relationships between variables and so I think there might be this is an ambitious project but there might be ways to actually create kind of uis like user interfaces that help people build generative models by like asking them a sequence of questions for instance do you want this variable to affect that variable and then depending on their answer you can kind of um pre-parameterize part of the a matrix or something and then the actual structure of the a matrix sketch window down to a sequence of kind of yes no questions about the different contingencies in the world another thing is interfacing with openai gym which we kind of already have done like there's a few examples where we've done this I haven't put these on the on the infra actively on GitHub yet but this is something that's an open this is like a very obvious and easy thing to do because like we wrote our environment class as if it was based as if it was a gym environment anyway so once you do that it'll open up to comparing active inference agents to all kinds of reinforcement learning algorithms on hierarchical models is a big one so um basically allowing you to stack hierarchically active inference agents within each other so like yeah there's a lot of temporal depth that you can get out by stacking active inference agents into hierarchical things so like one time scale of inference and planning is happening at a slower one slower than a a sub faster time scale we need more demos that demonstrate parameter learning so you can do updating of a b and d arrays I don't think you can update C so far and I know this is something Daniel you mentioned to me which is uh people in your in the active inference Institute are generally interested in um updating the beliefs about the generative model parameters basically and then there's things like sophisticated inference which is kind of a more recent version of planning under that's kind of interesting and has some computational benefits to it and then hand in hand with sophisticated inference goes this thing that people have developed have had to deal with in deep reinforcement learning for a while which is how do you tame combinatorial explosive policy spaces so when you're doing deep planning over time uh the number of policies is exponential

in the number of time steps that you plan in so that's there's various techniques for dealing with that like Monte Carlo tree search which I think some people like teofio Champions and others have already tried to start implementing their own implementations of palm DPS for um and so along these lines I want to just point out that we're actually very close to getting this working now so fitting Pi mdp models to empirical data so there's a branch that um Dimitri markovic and I have been working on called the agent Jax Branch we've basically written a back end for pine p and Jax which not only lets us use a bunch of statistical probabilistic inference techniques from like numpyro to invert or infer the parameters of Pi mdp Agents from like data for instance compute uh collected from Human participants but the fact that it's also back-ended in Jacks means that Pi mdp is now fully auto differentiable so it means you could stack a deep neural network layer onto the like before the a matrix layer of a pi VP agent and then you could use something like the variational free energy or any other objective functions to automatically train the parameters of a neural network that's linked to a pi VP agent so this I think just re-implementing it in back ends like Pi torch and Jacks is like a huge benefit because this will really allow you to extend pine P to much more High dimensional State Spaces by linking up deep neural networks to various components of the agent's body as you were describing it Daniel um and so we originally did this just to allow you to do fitting of empirical data but it comes with the side benefit of allowing you to differentiate and pass like back propagate gradients that you would use for updating deep learning models which is I think really exciting so that's like almost done I mean yeah like we're very close to putting up a notebook that actually does that if you look in the agent Jack's Branch now it's not very organized but that that stuff is now there and implemented another thing is nor Sajid and I have actually implemented some of the environments from her paper active inference demystified compared and we've actually done that with pi MVP in openai gym like in the frozen lake environment is a popular one for simulating B Matrix learning um so that's something that's also just like we we've done that and uh we need to like upload that or I don't know write a short paper do something with that because so there's a lot of like these different tendrils that have been explored it's just a matter of pushing forward and actually putting them up on the pine BP repo um yeah and then these other things I would like to find time to do but I just haven't but I mean as I kind of said in the beginning this was very much a collaborative effort so I don't also want to be um necessarily the one who's like doing all of this because I think it also is healthier for the development of the package if different people are kind of taking the lead on different things and developing it in their own way so that's something I

also just generally like to encourage is for all kinds of interested people to get involved in the development and I don't think Brennan is here uh but Brennan Klein also who's a post-doc um and research scientists at Northeastern University at the network Science Institute he started these Pi mdp fellowships so he got funding from Northeastern I think also the Templeton Foundation to fund people to work on Pine MVP development or prime DP adjacent projects I think the first round of applications is over but it would just be a this is a good opportunity to advertise that I think there's going to be another cohort in the summer so this is a kind of seemingly ongoing source of funding so it's just nice to see that other people are kind of trying to push Pine VP in their own directions so that's just an encouraging development that I want to keep everyone apprised of um oh yeah and then I'll just end by patient on the read the docs website which is really nice for creating Auto documentation and so we have a bunch of demos up there we have different tutorials um we have another a new demo that's not listed here which is about just calculating the variation of free energy in discrete categorical models which is based on a demo from Ryan Smith and Christopher white and Carl fursten's paper on uh unlike that big tutorial paper on active inference so I re-implemented one of the demos from that paper and now that's also in the docs um yeah so you can open all those demo notebooks in collab and just step through them um and they're really you don't need to have even python installed on your computer to use them you can just open the links in collab and step through the code and like build your own active inference agents so it's just useful for pedagogy that's why I mentioned if you're just getting started I would definitely recommend going to the to the documentation so yeah um thank you all for listening and for uh letting giving me the chance to to talk it was it was nice to be here as always and I guess like yeah we I I listed at the bottom for the next live stream we can go through some of the demo notebooks but we also could go through them now if there's time maybe we'll do discussion first and then just see there's time awesome thanks yeah let's address some questions from Daphne and Jacob I'll ask some from the live chat and then perhaps you could share one or a few of the examples on the read the docs and we could just look structurally at what the anatomy and physiology is of a notebook so first Daphne or Jacob any thoughts or questions yeah yeah go for it um I'm wondering on the on the Jacks implementation are there any requirements on defining the generative process at all or is it is it just about defining the structure of the generative model that we then fit to experimental data and I guess this also relates to another question I had in scaling these models to State spaces or general processes that we as modelers don't have the Liberty to actually Define ourselves but we want to deploy and train these

agents in general processes that are already out there like in an online setting where you get categorical or discrete data coming in yeah totally um for so on the side of that's a great question on the side of um say I had a plan BP agent that had a bunch of deep neural networks attached to it and I wanted to train it uh on in a deployed setting so it's like out there you know let's say it's an agent that's trading on the stock market or something it's like placing bets to buy like cryptocurrency let's say in that case in the same way to train a deep neural network on that kind of data you don't need to pass gradients through the generative process which of course you don't have access to you're trading on the stock market so in that sense no there's no requirements on passing gradients or or writing up a generative process that is also Auto differentiable there is one case when you would want that which is often in the case of empirically um fitting Pine DP models to data often one thing that you want to do is you have a bunch of like you basically have a history of actions and observations of a human participant you fit the model the parameters of the pi VP agent that best explain the observed actions of your participant and you know the observations because you're an experimenter who like decided this person is going to see the sequence of observations so you can do all that without having a differentiable generative process or environment but then there's something in in Bayesian inference that's called like a posterior predictive check where you say Okay given my inference about the uh the parameter of the pi VP agent then I'd like to roll out the expected behavior of this agent given the my best guess for what this agent's parameters are so that's called like a posterior predictive density where you say given my posterior um estimate about the agent's parameters what would it look like in the future under these posterior parameters and to do that in in using numpyro which is the probabilistic inference framework that uses Jax as a back end you would want to have a generative process that is also Auto differentiable um but in that case I expect that writing those generative processes would be easy because that would be in the case of fitting a human's Behavior to experimental data where they're like in a controlled task environment so if it was the case of trying to fit someone's parameters like the the value of their C vector and they're performing that scene construction task where they're sick cutting around you could write the generative process because you as the experimenter or developed the psychophysical like task that they're interacting with you could write that also in Jax when you're doing the modeling so that when you're doing these posterior predictive checks you know that that's also written in Jax and that you can compute those quantities but in a deployed setting you're not going to even be able to do any kind of posterior predictive check in the future because you don't know how the

environment actually works right so you'd have to you that that wouldn't even be something that you tried to do in the first place um but yeah so there's nothing inherently stopping you from just as long as the models are differentiable in the same way they are with deep neural networks there's nothing stopping you from just throwing them into an environment where you don't know how the rules of the world work yeah the inexorable logic of natural selection or free energy minimization or just non-equilibrium systems whether or not they know what's out there either it's going to work or it won't and if it fails it fails and the computational environment allows us to exist in this kind of Gray Zone where the computational agent might be quite poorly adapted to a given deployed setting but the computer program will still run but of course we're interested in cases where the computer program runs and the agent is able to event some kind of meaningful or even useful Behavior exactly like we could we could all imagine very simple Prime DP models that would do terribly in some tasks right just like a dumb model that has two hidden states that it believes just stochastically switch between each other and then you give it the task of uh in making investments in like a 10 stock portfolio and of course its model is not fit but the the promise of applying big deep function approximators to different ends of the Prime DP agent means that hopefully you could then learn a good generative model and then still combine that with some lower dimensional generative model up top that can do all the nice inference and planning with active inference but it can deal with high dimensional or ugly hard to tame observation and action Spaces by using deep neural networks so I think that's really the way to to just in the same way deep learning has gotten that to work in a lot of cases this is the way to kind of do it with with prime DP models as well Daphne any remarks or I'll ask a few from chat um I don't really have any questions but I do think that it is really fascinating and like I think it's really exciting to think about yeah like as you said like learning the generative model of like an agent learning its own generative model given like some real world data to like figure out like what is them and I guess like in terms of what you were talking about with like the function approximations and numpyro and stuff is that still like what like are you guys still working like from that code base and then finally convert those or have it kind of started from scratch yeah we we pretty much started from scratch the pi DCM thing I'm actually not sure what the like the IP status of that is because we worked on it as part of nested mine so I I don't have access to that code anymore but that was more implementing like variational LaPlace in which you were we worked on that together with variational Applause injects which is the way that you do gradient descent on free energy when you're trying to do um inference what we're doing now instead of that is we're saying

can you rewrite a pine DP such that you can pass gradients through it Jacks like accelerated gradients and then use numpyro to do all kinds of fitting routines not just variational LaPlace but you could use mcmc you could use like numpower just has a massive you know library of different uh probabilistic approximate Bayesian inference techniques so you can kind of throw the kitchen sink of numpyro inference techniques at a prime DP model so the challenge there is just rewriting a prime DP model so you can Define like a likelihood function that goes from like the Palm DP parameters to the observations which in this case would be the actions of the agent and in order to do that in a way so that can like play friendly with Jax we just had to make sure that all the interior functions of the pine DP agent like the inference the planning action selection all of that was um written in Jacks so that you can pass gradients through it when Computing like likelihood gradients effectively yeah that makes a lot of sense all right I'll ask a few questions from the live chat so first most descriptions of active inference across the literature are written in terms of matrices but Pi mdp clearly works with tensors do you have a good reference for how the operations are different when generalizing the equations from matrices to tensors that's a really good point um this is one of the things that actually frustrated me a lot when I was first learning about active inference was I I noticed exactly what this person asked is that a lot of the basic operations are written as if there's only single dimensional hidden States and single dimensional observations so everything is like they said Matrix Vector products and Matrix math but what we're really doing is tensor multiplications and tensor products so in terms of references for how that works um there's yeah so essentially there's nothing super qualitatively different did these tensor operations that we do in pine P are basically just fancier ways of expressing sums of matrix multiplication so there's the mathematics of it is all still standard linear algebra is just the way we represent these high dimensional matrices as tensors is just a more efficient representation so mathematically it's nothing too crazy the way I learned about how that worked was just by staring at functions in Matlab for like a year until I just figured it out but it wasn't easy and there's definitely better options out there now um so um one reference off the bat I'll recommend is is the appendix appendices um Pi mdp uh the paper the archive paper so that that like I think appendix a or in all those appendices basically deal with the full tensor factorized version where it's not dealing with matrices but we're actually indexing into higher Dimensions another one which I think um originally discusses the tensor products and the tensor factorization is in an appendix of I think active inference curiosity and insight which is where they first talked about like novelty and parameter Information Gain that's a paper um I'm sorry I don't remember the year it came

out it's either 2017 or 2018 but I know the paper title is called active inference curiosity and insight and in one of the appendices they actually do the full tensor-based mathematics and then finally another good reference is a recent paper that was I think headed by teofio Champion I'm just gonna try to find it real quick because it's I don't want to forget this maybe I'll stop sharing my screen did I stop okay it stopped it's a really good reference that has appendices about um doing tensor math for active inference in particular we also recently learned about the branching time active inference which speaks to some of those questions of computational complexity and and all right yeah I should have mentioned that that's like probably the most promising approach to date about the um uh to to finessing the computational complexity of active inference so yeah this this one is by teofio Champions Mark gresh I guess that's one of his advisors and Howard Bowman his other advisor and that is called multimodal and multi-factor branching time active inference which I just posted so I haven't read this myself but I've heard from other people like I think Alec chance told me that the appendices are really good for the tech the full tensor generalization of all right awesome well I've added all of those um citations mentioned into the YouTube live chat awesome thank you I'm going to ask a following question from Fausto using a Jax backend makes it easy to wrap pymc3 apply MC3 around it EG to have pi mdp as an operator to use in a pi MC3 model is there any plan to do this I ask because there's a growing Bayesian community in Python around Pi MC3 um that's really interesting I didn't know actually that Prime G3 was also had a Jack's back in so um I'm I don't know to be to be honest I'll start by saying that I don't know because my introduction to like probabilistic modeling in Python was through DME Dimitri markovic who basically sold me on numpyro numpyro is the way of the future and I know numpyro has a Jack's back end and I think numpyro and Pi MC3 occupy a similar place in that ecosystem of probabilistic inference in Python um I don't know how models are specified in pi MC3 I am assuming it's not too dissimilar from how it looks in numpyro and because all the low-level back end is now written in Jax I I can't promise this but I would assume that you could just write a pi MC3 model and the same we wrote way we wrote a numpyro model that wraps um Pi mdp functions but only the pi MVP implementation injects so if Pine Z3 only depends on Jax at the low low level then yes it certainly can work but um I don't know is there anyone else here who has experience using pine G3 and might know because I just don't know enough about it I used YMC a little bit but I think it is like yeah as you said like quite similar to numpyro I think that probably you'd be able to do the same things that you're doing um with numpirers like integrating on pyro at with pi MC3 as well ah cool and so it's it's primary um

back end is Jax is that um I didn't actually do that um I didn't know that either yeah in comparison with the matrix multiplication of Matlab what makes you excited about the probabilistic programming Direction and all of these packages and approaches that we're naming how does the probabilistic programming differ from just writing out the matrices and calculating them on paper and why does that have some promise for implementing active inference models um I think the biggest advantage of the probabilistic programming is not necessarily for simulating active inference agents for sure the Matrix multiplications are sufficient having it in Jax makes it much more scalable so you can use all the vectorized operations to run like tens of thousands of active inference agents simultaneously because you have highly optimized just-in-time compiled functions and Jax that allow you to it just basically speeds things up an order of magnitude but the probabilistic programming angle is not as much for simulating active inference processes as it is for doing inference or fitting models of active inference agents to empirical data so say observe an animal or a person doing something now what we can do which we can only do an SPM but we can't do in Pine VP yet until now is we can take a sequence of someone's Behavior and then infer the best um parameters of an active inference model that explain their behavior so given how someone decided I can say oh their a matrix must look like this or their C Vector must have this um Precision to it like you could infer someone's risk sensitivity or uh their risk aversion given their behavior and the nice thing about being in probabilistic programming languages is there's so many different methods that haven't been really well explored for fitting active inference models in the Matlab literature because almost everyone there uses this um variational Bayes approach where you basically minimize free energy you use a gaussian approximation for the posterior it's a very specific type of variational inference now that it's in numpyro or perhaps Pi mt3 again this will be very soon it's not fully implemented yet we'll be able to throw all different kinds of probabilistic inference techniques that have their different advantages and disadvantages like a big thing in probabilistic inferences is um mcmc Monte Carlo Markov chain Monte Carlo inference it's supposed to give less biased uh uh posterior distributions like there's advantages to using mcmc over variational approaches to approximate base and inference and one thing that I haven't seen done which I would love to see and a lot of people in the computational Psychiatry Community have have complained and told me what they'd like to see is like a side-by-side comparison of variational ways to infer Pi mdp parameters or Palm DP parameters versus like mcmc approaches so once you have everything in a probabilistic program framework you can do side by side comparisons between all the

different inference techniques that you wouldn't necessarily have um if you were limited by a language where only at one or two or three inference techniques are implemented so it's basically just taking advantage of all the work that people have worked on numpyro have done in implementing all these different kinds of uh inference methods yeah awesome and fausta followed up the primary back end uh for YMC is a Sarah but the new version can use Jax I might get around to writing the pi MC3 wrapper once the Jax version of Pi mdp is stable it sounds very doable well that's awesome if you see it as likely and you have the affordance then just minimize your free energy and you won't be surprised when you do absolutely that's great that's promising nice um Jacob or Daphne or I can ask another question the thing for me right now well you mentioned message passing several times in the context of Pi MGP so what is message passing and how was it used in pi mdp it's a good question um so message passing in general describes like a set of algorithms that you can use to do an exact or approximate Bayesian inference um so oftentimes in in the to make it very Concrete in the in the context of doing Bayesian inference about hidden States so what an active inference agent will have to do when they're faced with some observation they'll have to combine messages like one message corresponds to the message the sensory information and then another message corresponds to their prior beliefs about the world and they use some algorithm to combine those messages to optimize a belief about the current hidden state of the world so in pion P we have a very naive kind of computationally efficient doing way of doing that which we just call naive or vanilla fixed Point iteration which is like the most simple message passing scheme you can think of which is just I'm actively filtering hidden States using my priors from the past so I say given where I was at the last time step where should I be now given my B Matrix and then I just essentially combined that with my incoming sensory message which is just the observation passed through the likelihood Matrix the a matrix and then I just combine those two messages together and that resulting thing is my posterior distribution my posterior or best belief about hidden States that's like the simplest form of message passing that has this very temporally shallow current evidence combined with prior to form the new belief it has this very kind of Bayesian flavor to it right where the best posterior is just a product of the likelihood and the prior there's also more advanced message passing techniques that are used when your beliefs themselves are more complicated so in the full implementation of active inference that's in the Matlab version agents don't just have a belief about what the current hidden state is they have a full predictive and post-redictive kind of um tensor or cube of beliefs about all the hidden states in the future and all the hidden states in the

past further condition on all the policies I will potentially take or could have taken in the past so you have this massive like belief tensor uh that will stretching into future and past hidden States and uh further factorized by policy and when you have that kind of belief that you need to update you have to use more sophisticated message passing techniques one of which is called a marginal message passing there's something called variational message passing and all these different message passing techniques are just essentially consistent in passing messages forwards and backwards in time as well as across different variable that characterize the hidden state which we call Hidden State factors and and the message passing algorithms are basically still amount to combining sensory information with prior beliefs but they just have kind of more complicated trajectories through the space of beliefs in the future in the past there's I'm there's people who can explain this much better than me I've implemented some of these in pi mdp but I would refer people to there's a really nice um paper I think you may have retweeted it the other day Jacob it's about it's called mean field oh yeah um the paper comparing the mean field and that they approximation and the best air approximation neuronal message passing using mean Fields Bethy and marginal approximation par Markovich kibel and friston 2019 um it's a paper that a few of us have been walking through looking at how the different free energy functionals look different under different approximations and it'll probably be a focal paper in 2023 for us to really dive into because a lot of these um vintage let's say 2011 to 2019 papers now packages and development directions such as Pi mdp are facilitating [Music] these methods to be actually used and there's a huge wealth of conceptual possibility it proposed heuristics exciting use cases relevant other kinds of connections and as you brought up earlier it was very Matlab bounds to bring those kinds of connections into the last mile and then especially the more granular or modular developments were Under the Umbrella of the SPM vbx which prevented them from being meaningfully shared in a true distributed open science or decentralized science way and so that's why of course we've been so excited to work with and build on Prime DP and learn about it more because this is exactly the kind of composability of active inference agents and their different implementations that is going to be able to be worked on in a massively distributed way somebody might specify a really interesting a matrix somebody else might specify an interesting B somebody else is going to link those together into a new kind of agent someone else can implement it differently and so it brings like a natural it kind of factorizes the process of developing these algorithms which previously were almost always either fully Matlab or and or bespoke and very custom and fit for a given paper but not necessarily adaptable along the relevant axes that one would

want to use for a modern especially pythonic setting totally I can agree more I mean that's a great way of thinking about it too is like I'm making modular flexible code that exists in the ecosystem of other packages you're essentially factorizing the collective Minds representation of the task at hand or then different parts of that representation can be worked on without having to pass messages or take into account what's going on across the entire network of distributed workers so like someone can write their own you know even better message passing algorithm and then just slot it in to use with pi MVP without having had to learn about how every little facet of prime DP works you know so yeah that's a really important I think thing about just open science and modular software nice well in our closing minutes of course Jacob or Daphne any remarks or questions and also any appetizer what kind of models are people excited about and or what might mean we see in the following live stream which will be in January 2023 model stream 7.2 um I would just say that I think that the notebooks are really really useful so like it's a really great resource for people who um are trying to build and model and understand like what's going on under the native IMDb and I think that it would be really cool to have an extension to those notebooks that also talks about learning the duration parameters for the A and B matrices I think that that would be really really cool and thank you so much Connor you wanted to talk yeah thank you for coming that's a really good point and this is something that Daniel also said um earlier in the email is that updating A and B is like it's very very under documented right now and I think that would be because that is a form of learning the generative um that right now we don't it's not the most sophisticated way you still have a fixed number of rows and columns so you make some assumptions but that is like a flexible way when the agents are themselves learning the b and a matrices um yeah we should definitely maybe that can actually be a I I can just add that into the notebook that I was planning on showing this like a kind of epistemic two-armed Bandit task we can just add in some A or B learning to that and just show how that works um I would make a new notebook that uses that cool eventually for the textbook and for every paper it would be amazing to be able to see the code the analytical representation a graphical representation and different natural language representations because they're all formally uh formally connected and they could all be rendered as such and that would really one might expect increase the accessibility and rigor of a model and help us compose and connect across different domains and just welcome and recognize many different kinds of learning and modeling absolutely jakov any um no questions at this point but uh also just thanks a lot for the awesome presentation and I'm really excited about all the emerging Integrations and use cases that will that will undoubtedly

uh spring up we've been uh we started exploring uh numpyro as well and kind of discussing how um that can be used for scalable active inference models and I'm really excited how primevp will interoperate with all of these different Integrations and it will be very um exciting to use Connor any penultimate words um I guess maybe just in in the spirit of what you were saying I can just show a skeleton of what we could go through next time oh great all right we see it um yeah is that is that okay it looks great looks great yeah basically this is a collab notebook so I encourage anyone to go to the um Pine DP tutorial website and each of the notebooks like Daphne was saying they had their I'm pretty useful and they have collab links associated with them um and you can just open the link and then explore this one I think is part of the agent API yeah this is the same one except I showed this one recently at the course on computational Psychiatry so I've updated a little bit so I can share this with um you Daniel and then you could either put on the Discord or wherever this one's a little bit more updated but the basic one here will still that you can access here will still show the same thing but um in any case the thing is you just open up collab you need a Google account that's the one limitation to use these um you can install locally infra actively Dash piondp uh import numpy matplotlib there's just some imports here and then this whole this the spirit of this notebook is essentially just going through all the steps in setting up the generative model so creating your hidden State factors that's something we didn't really talk about is factorized representations in the context of Pi mdp building the B array which not only has um like you can just do it yourself but there's also these hidden cells with solutions to each of these things and then the main brunt of this notebook before running active inference is just stepping through and actually initializing um the entries of the a b c and d arrays and next time I have some slides to go along with this so we can basically go between the slides and the actual code same thing with the a array so you like have uh some representation of what you want the a matrix to look like and then you go into the code and actually build it out and you do that sequentially for each of the components of the generative and you're plotting them along the way so you can see what it looks like after you've built it and then we actually uh we actually Implement after you've built the general model then you actually plug it in to an active inference agent so this this the first stuff is building a factorized a and b for a generic um General that I think is just a more sophisticated grid world but then we actually that's kind of the introductory task and then we go in and we actually build the A's and B's and c's for this epistemic two-armed banded task which is basically just like a teammates and then you uh build that out you know you're writing in all the little sub matrices of your a matrix that's why

there's so many cells like I said it's like that's the longest part it's actually building things out the C Vector basically the reward function which has Daphne was saying earlier you're actually encoding in and then finally you basically do those those steps that we discussed during the presentation you just plug all your painstakingly generated A's and B's hopefully not too painstaking into your agent class you generate the two armed Bandit the epistemic to arm bandit uh environment which is just the rules about how the world Works given the agent's actions and then you actually run this active inference Loop which is as we discussed just effectively running a loop over time doing hidden State influence policy inference action sampling and then stepping the environment to get the new observation and then at the end I've just written this like helper function that can basically plot the history of choices and beliefs um so then you at the very end this is like the more a fun experimental part you can mess with the parameters of the uh environment and also the parameters of the agent's model and then start seeing how that changes behavior um just by kind of iteratively running active inference simulations and plotting the resulting Choice behavior and history of beliefs so that's the general that's a little preview I guess of what we can do and we can also have a little sub module in here we're actually letting the agents update their beliefs about the a or the B Matrix which could be cool awesome looks really exciting and on a final SPM note in the SPM textbook and in experiments sometimes there are these incredible grayscale matrices that summarize like multiple experimental factors across 100 participants and uh so it's really interesting to see how you show with also that black and white or grayscale Matrix representation and how that provides a visual feel for some of these topics that we've been discussing it and of course the representation is formally linked with the Matrix but sometimes just saying well you have two options and there's ten states in the world and the likelihoods look like this instead of seeing that as like a spreadsheet with numbers seeing them in the grayscale provides kind of a feel and it looks really nice and so it looks like an awesome session we'll have for DOT two yeah totally that's interesting that you bring that up like that's something I've just always been doing and I think it's very much because I learned all of that from Reading those active inference and SPM papers um so I very much just kind of borrowed that visualization technique from them um and I kind of took it for granted but yeah it is interesting it's clearly not the only way to go but I always just found it very intuitive to think of probability you just kind of can color it use colors because the numbers are too specific it's the color the grayscale that visual aspect really like kind of just shows this thing is more likely than this thing yeah yep cool all right well Daphne and Connor thanks a lot for this awesome

session and we'll see you in a little bit more than a month for the dot two great thanks so much
Daniel and thank you everyone peace take care bye thanks everyone